earns 300 points, then they should obtain a status of Silver." Once you have this test working, you could proceed to implement the real version of this class.

This test is now totally independent of any particular implementation of the `StatusService` class. The test will run quickly and require no setup, and if it fails you'll know that the problem is likely to be in the `FrequentFlyer` class and not in the `StatusService` class.

Of course, this now means that you need to write a separate specification for the `StatusService` class. In the next section you'll see an example of some of these low-level specifications.

10.3.7 Expand on low-level technical specifications

In the previous section, you saw how mocks can be used to define contracts between classes. After these contracts are defined, you need to come back to ensure that your service class respects this contract. Likewise, in section 10.3.5 we discussed using a minimal implementation of the status service, but not one that would be sufficient for the full acceptance test. In both cases, you need to come back to complete the implementation.

Suppose you're implementing this as an in-memory service. You might start off by defining how this service behaves for zero points:

```
StatusService statusService = new InMemoryStatusService();

@Test
public void should_stay_on_bronze_for_zero_points() {
    assertThat(statusService.statusLevelFor(0), is(Status.Bronze));
}
```

The status level for zero points should be Bronze.

Low-level specifications often cover cases that aren't included in the acceptance criteria, such as boundary conditions like this one. Acceptance criteria are high-level examples that don't need to cover every possible input and expected outcome, only the ones of significant business value. Often you'll have more technical edge cases that you'd like to document that aren't included in the acceptance criteria.

Once you've implemented enough code to make the code work for zero points,[3] you might proceed to another edge case:

```
@Test
public void should_stay_on_bronze_up_to_299_points() {
    assertThat(Status.statusLevelFor(299), is(Status.Bronze));
}
```

What is the status level for 299 points?

Then you might proceed to one of the acceptance criteria examples:

```
@Test
public void should_earn_silver_for_300_points() {
    assertThat(Status.statusLevelFor(300), is(Status.Silver));
}
```

What is the status level for 300 points?

[3] We won't dwell on the implementation details here, but you can find a sample solution in the sample code for this chapter.